

AI BlogNest – Project Document

Generate High-quality Blog Content Using AI

Project Overview

AI BlogNest is a modern REST API CMS built with Node.js, Express, and MongoDB that enhances traditional blogging with Generative AI. Integrated with Google Gemini AI, it automates context-aware content generation and summarization. The platform is fully secured using stateless JWT authentication, Role-Based Access Control (RBAC), bcrypt hashing, input sanitization, and rate limiting.

Team Members

Team ID: SWTID-2026-8198

Class: III B.Sc Computer Science, Shift - I

Name	Email	NM ID
Ajaykumar N	ajaykumarn779@gmail.com	89C689CD858133CAC7E435296315E5F4
Sharun C	sharun.chandran200715@gmail.com	7E08523D9521B62AD5648440FB9B2992
Shrihariharan R	shrihariharan15@gmail.com	DEADA76449866BD87694A6DA72308B29
Sridharan N	itzmehsrisri@gmail.com	A39D31EC73E063CA40841F026876EC17
Sriram D	srisrisrisri5028@gmail.com	B21EE771CF2FCBC94E24057207C40BBA

Phase 1

Brainstorming & Ideation Phase

Phase 1: Brainstorming & Ideation Phase

Background

Priya is a content creator who manages multiple blogs but struggles to organize posts, schedule publications, manage categories/tags, and track reader engagement efficiently.

Problem

- Difficulty managing multiple drafts, categories, and tags.
- Limited visibility into post performance and engagement analytics.
- Time-consuming processes for content approval and comment moderation.
- No streamlined system for handling media uploads or metadata.
- Lack of collaborative permission tiers for editors, authors, and administrators.

Solution

The BlogNest backend delivers a centralized, secure API wrapper that decouples application business logic from frontend delivery systems. It introduces an internal permission matrix controlling specialized services for content scheduling, category taxonomy, media path registration, and granular user roles.

Usage

- **Author Usage:** Creates blog posts, uploads media pathways, organizes content drafts, and reviews their distinct post-performance data.
- **Editor/Admin Usage:** Accesses protected endpoints to review content queues, change publishing statuses, moderate visitor comments, and orchestrate user privilege upgrades.
- **Reader Usage:** Queries public list routes to consume posts and invokes comment creation endpoints.

Outcome

By offloading workflow controls to a robust backend, content execution delays are minimized. Data structural integrity is strictly maintained through schema validation, security filters guarantee that only authorized roles perform administrative shifts, and persistent tracking provides raw analytics to measure post optimization metrics accurately.

Phase 2

Requirement Analysis Phase

Phase 2: Requirement Analysis Phase

Software Requirements

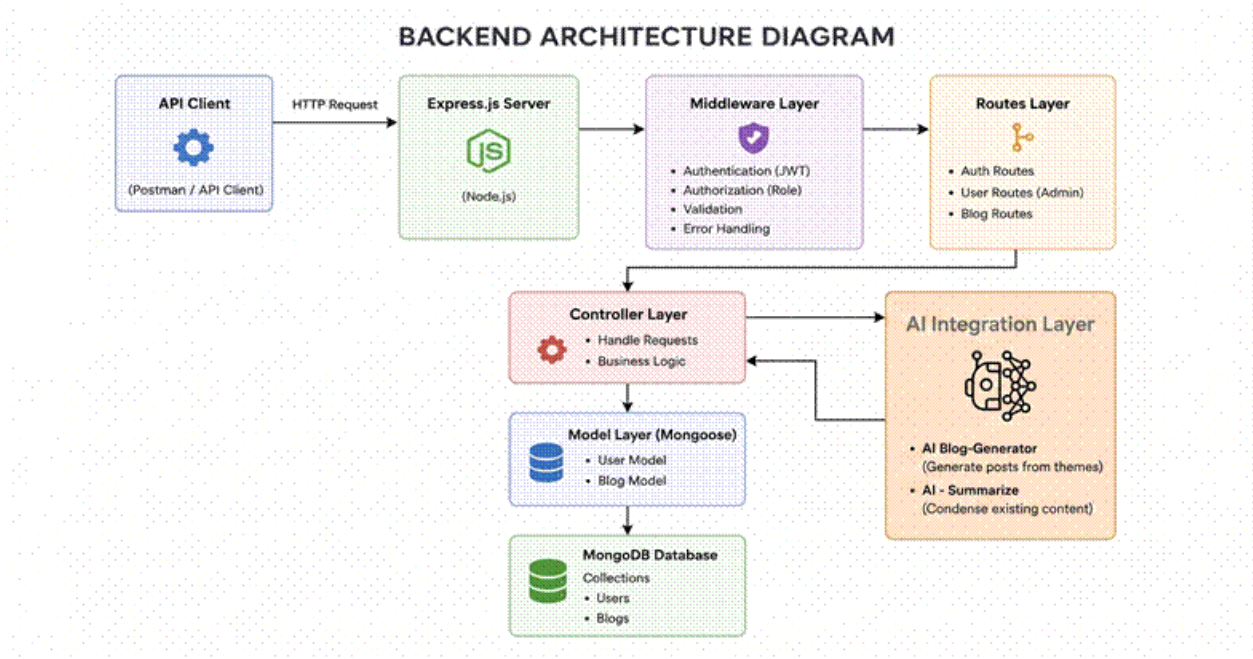
- **Operating System:** Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Express.js:** Lightweight routing web framework to construct backend RESTful entry points.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

Phase 3
Project Design Phase

Phase 3: Project Design Phase



The BlogNest application exposes structured endpoints handled by a layered server architecture model. Incoming requests undergo filter processing before interacting with data services.

Components Definition

- **Interface:** API tools like Postman query distinct route paths to extract JSON entities.
- **Express Server Gateway:** Monitors network ports, maps explicit HTTP methods (GET, POST, PUT, DELETE), parses JSON payloads, and handles CORS headers.

- **Authentication & Role Matrix Middleware:** Extracts tokens from headers, validates user context signatures, and evaluates if the specific role satisfies authorization thresholds for guarded resource paths.
- **Granular Services (Logic Modules):** Enforces isolated domain rules for blog lifecycles (Draft, Pending Approval, Scheduled, Published), comment queues, and system reporting indicators.
- **Database Interface Layer:** Mongoose translates operational application logic into performant native MongoDB queries.
- **AI Flow (Service Integration):** Provides intelligent content automation, including automated blog post generation and real-time content summarization. This layer connects to external AI models via specialized endpoints (/api/ai/generate-blog and /api/ai/summarize), allowing the controller layer to offload complex text processing tasks and enhance content creation workflows.
- **BlogGenerator with AI:** A specialized service that automates the generation of blog content, which integrates with the AI Flow service to streamline the drafting process.

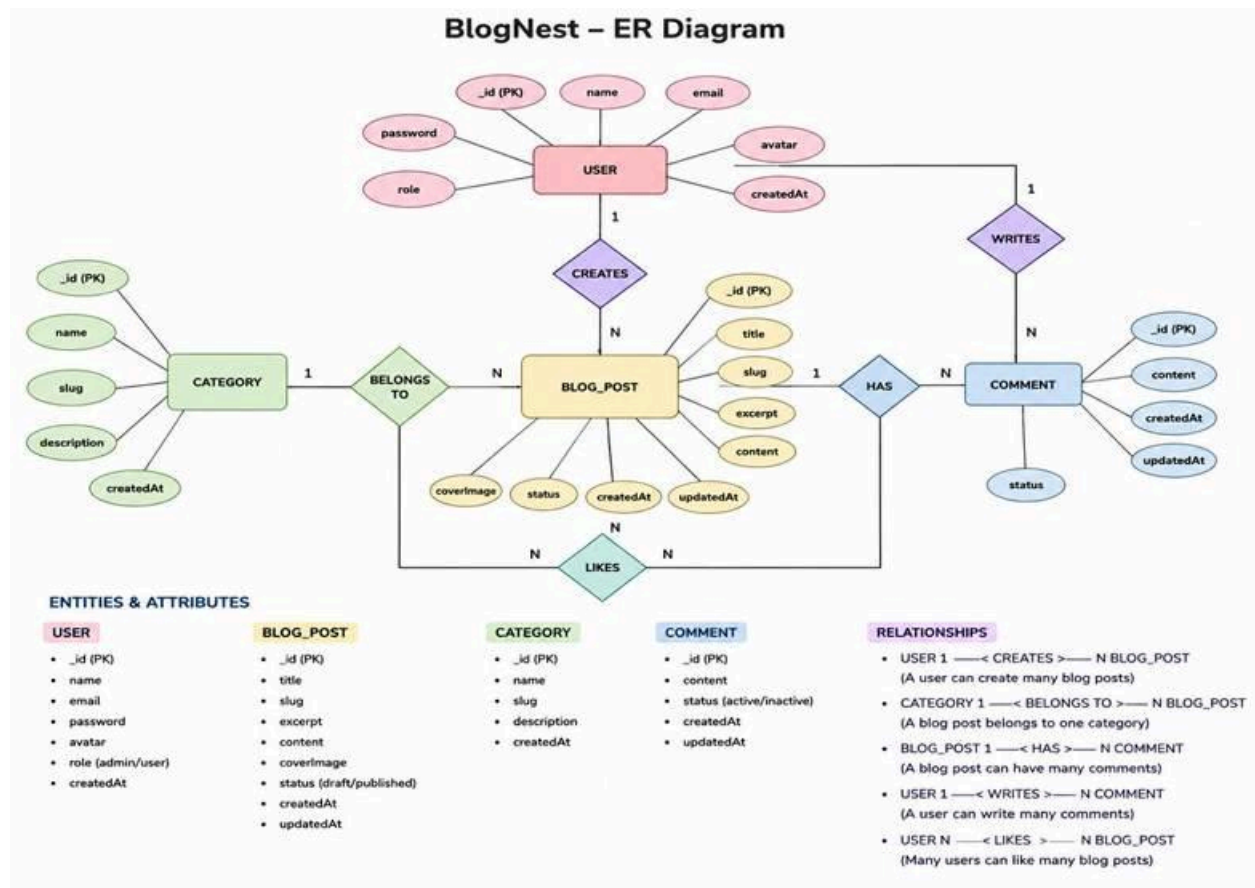
Phase 4

Project Planning Phase

Phase 4: Project Planning Phase

Entity-Relationship (ER) Diagram Description

The underlying document structure manages several collections bound by logical application references:



1. User Entity

- _id (Primary Key - ObjectId)
- name (String, Required)
- email (String, Required, Unique)
- password (String, Hashed, Required)
- role (String, Default: "reader")

2. Post Entity

- `_id` (Primary Key - ObjectId)
- `userID` (Foreign Key referencing User Collection)
- `name` (String, Required - represents Title)
- `photo` (Object/String pathway link)
- `message` (String - Core content body)
- `likes` (Number, Default: 0)
- `timestamps` (True)

Key Structural Mappings

- **User to Post (One-to-Many):** An author can submit multiple blog documents.
- **User to Comment (One-to-Many):** Readers/Users can append multiple response comments across independent post nodes.
- **Post to Comment (One-to-Many):** A single blog document stores individual referenced comment submissions.

Key Features

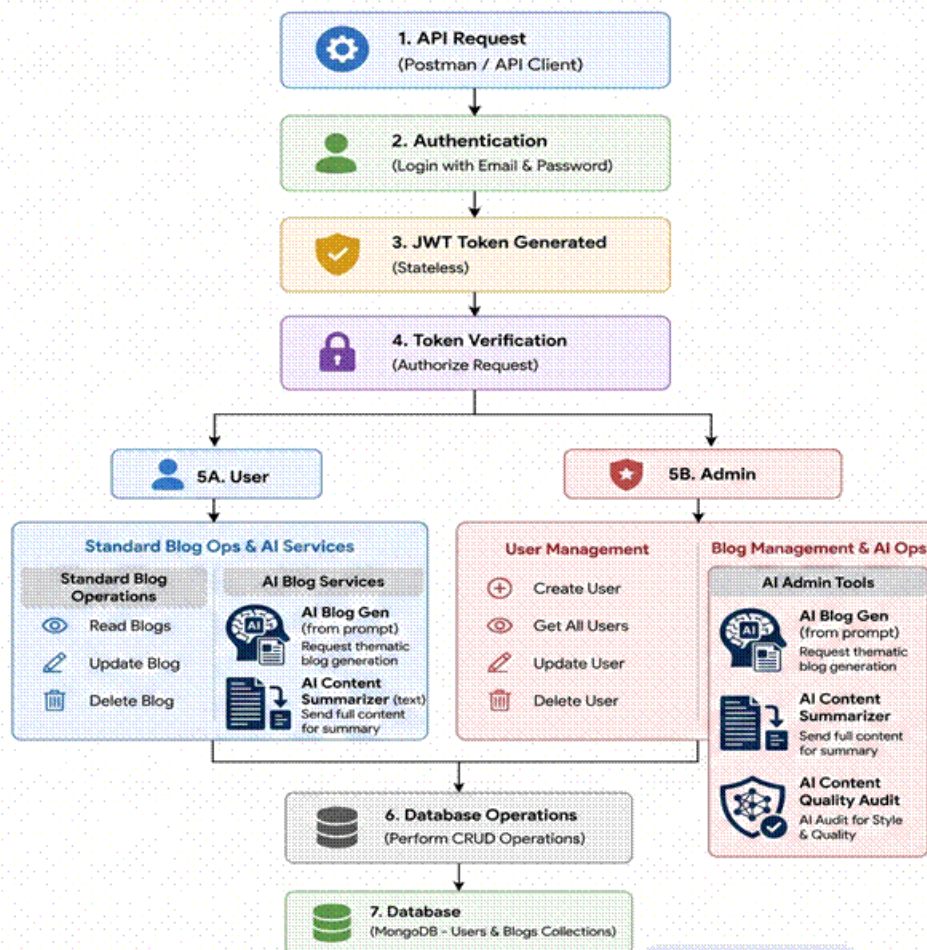
- **Role-Based Route Guarding:** Strict token processing blocks unauthorized tiers from modifying core settings.
- **CRUD Blog Execution Engine:** Complete schema management over content definitions, status states, and asset assignment paths.
- **Categorization System:** Native tag indices facilitate filtering and querying for enhanced discoverability.
- **Comment Moderation Logic:** Protected entry points for updating moderation flags (approved, pending, spam).
- **Sanitization Safeguards:** Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.
- **I-Driven Blog Generation Pipeline:** Integrates external Large Language Model (LLM) abstractions and context-aware prompt templates to synthesize full-length, structured blog posts from high-level thematic inputs.
- **Intelligent Content Summarization Engine:** Leverages semantic compression layers to parse extensive article payloads, dynamically generating concise abstracts and automated metadata summaries.

Roles and Responsibilities

- **Admin:** Complete environment override control. Can manage user database registrations, alter role assignments, prune malicious comments, and inspect full logging entries.
- **Editor:** Focuses on editorial workflows. Reviews pending draft posts, marks records for instant publication, and handles category cleanup scripts.
- **Author:** Restricted account access. Can issue creation statements only for posts bound to their explicit account record.
- **Reader:** Consumer access profile. Reads published materials and posts comments under an ongoing authenticated identity.

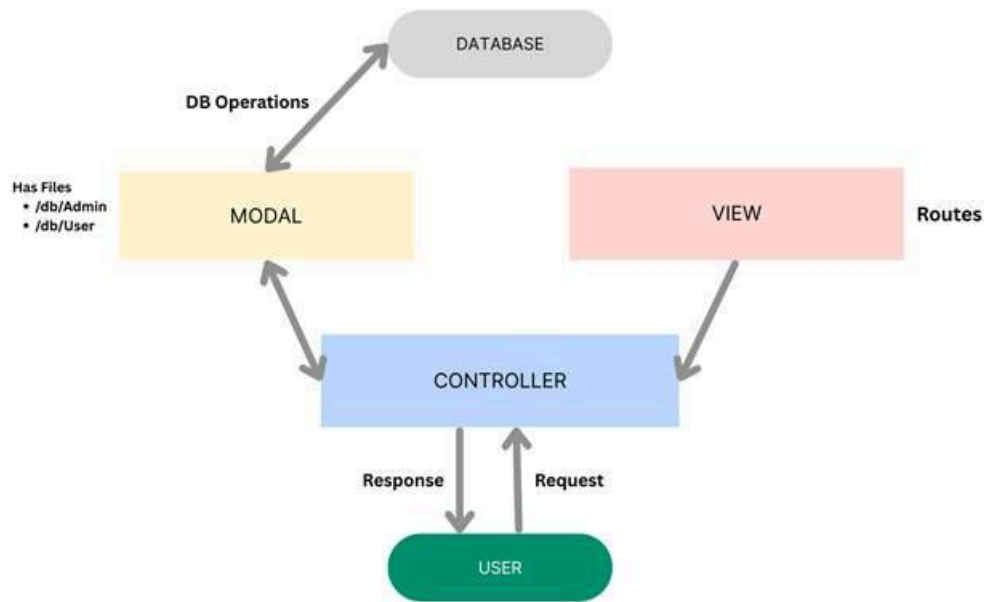
User Flow:

AI-BLOG USER FLOW DIAGRAM (Backend Only)



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structure blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets default states, indexing rules, and transformation parameters before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from the execution routes, checks structural request constraints, delegates processing workflows directly to models, and packages the resultant output into standard JSON return arrays.

View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API **Routing Layer**. It links network requests on specified paths directly to their associated controller handler logic maps.

Phase 5

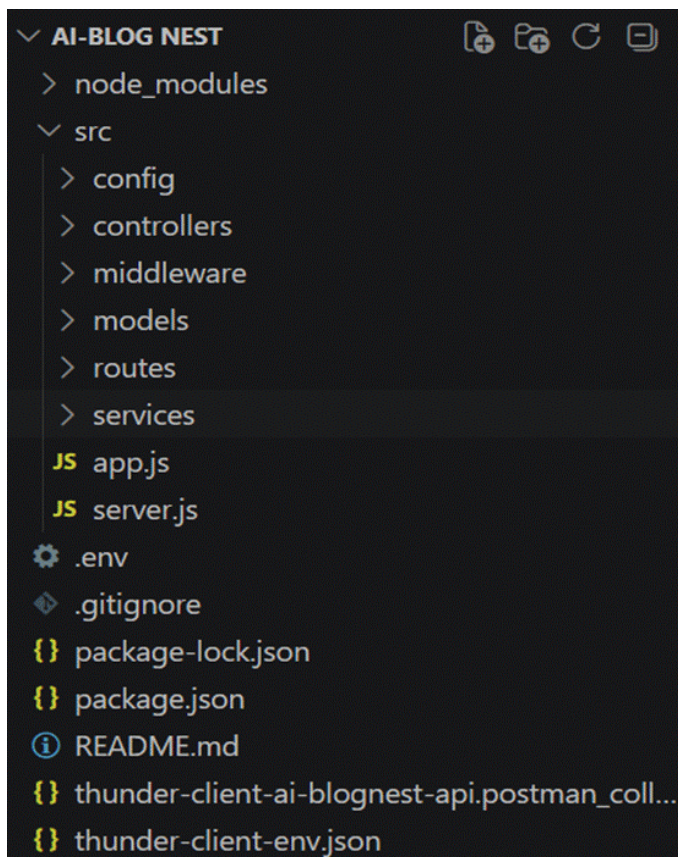
Project Development Phase

Phase 5: Project Development Phase

Covers initializing the Node.js environment, organizing the MVC folder structure in VS Code, and building the Express.js server. It details configuring the MongoDB connection using Mongoose (db.js), defining data schemas (User.js and Blog.js), setting up authentication and error middleware, and integrating Google Gemini AI services.

Creating project folder

1. Create a new folder with your <project name>
2. Inside that folder create one new folders.
3. Name another one as Server.
4. Now open that folder in VS Code.



Server setup

- Open Server folder in terminal of VSCode.
 - npm init -y
- Create files:
 - server.js
- Create folders:
 - models
 - controllers
 - routes

Backend Development

Server.js

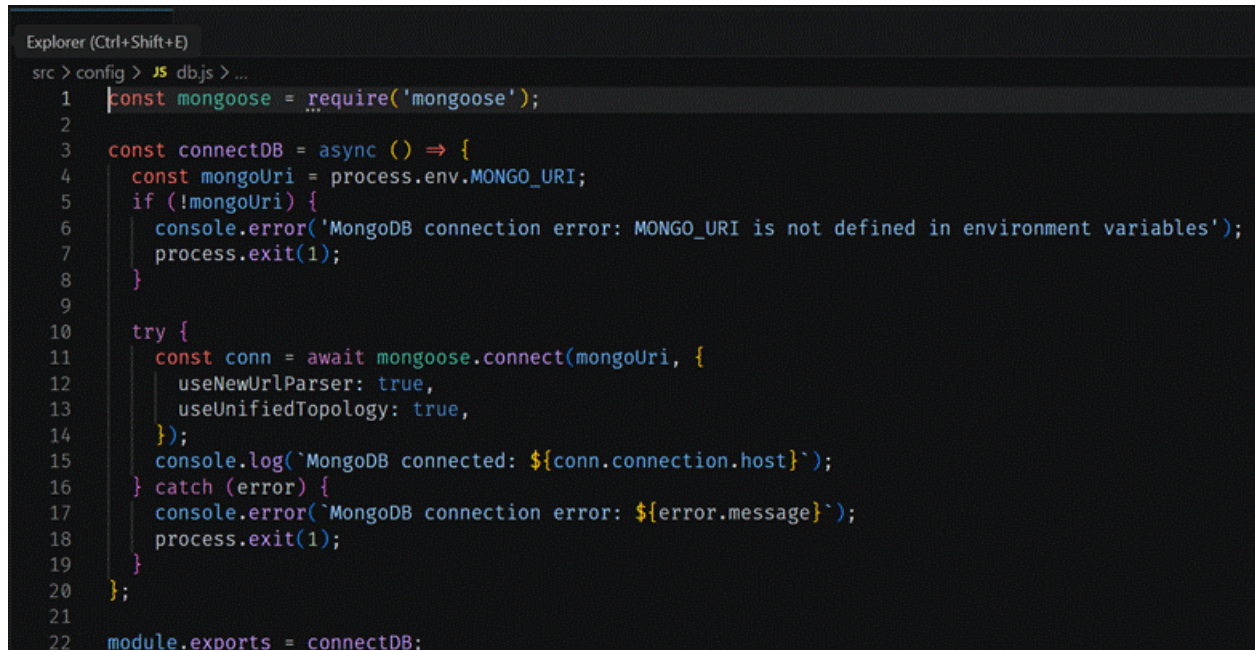
```
JS server.js ×
src > JS server.js > ...
1  const dotenv = require('dotenv');
2  const app = require('./app');
3  const connectDB = require('./config/db');
4
5  dotenv.config();
6  connectDB();
7
8  const PORT = process.env.PORT || 5000;
9  app.listen(PORT, () => {
10 |   console.log(`Server running on port ${PORT}`);
11 | });
12
```

App.js

```
JS app.js ×
src > JS app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const morgan = require('morgan');
4  const authRoutes = require('./routes/authRoutes');
5  const blogRoutes = require('./routes/blogRoutes');
6  const aiRoutes = require('./routes/aiRoutes');
7  const errorMiddleware = require('./middleware/errorMiddleware');
8
9  const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22 |   res.json({ message: 'AI BlogNest API is running' });
23 | });
24
25 module.exports = app;
26
```

Database Configuration

DB Connection:-

A screenshot of a code editor window titled 'Explorer (Ctrl+Shift+E)'. The file path is 'src > config > JS db.js > ...'. The code is a JavaScript file for connecting to MongoDB using Mongoose. It starts by requiring Mongoose. Then, it defines an asynchronous function 'connectDB' that checks for the 'MONGO_URI' environment variable. If it's not defined, it logs an error and exits. If it is defined, it attempts to connect to the database with specific options (useNewUrlParser: true, useUnifiedTopology: true). It logs the connection status and catches any errors, logging them and exiting. Finally, it exports the 'connectDB' function as the module's exports.

```
1 const mongoose = require('mongoose');
2
3 const connectDB = async () => {
4   const mongoUri = process.env.MONGO_URI;
5   if (!mongoUri) {
6     console.error('MongoDB connection error: MONGO_URI is not defined in environment variables');
7     process.exit(1);
8   }
9
10  try {
11    const conn = await mongoose.connect(mongoUri, {
12      useNewUrlParser: true,
13      useUnifiedTopology: true,
14    });
15    console.log(`MongoDB connected: ${conn.connection.host}`);
16  } catch (error) {
17    console.error('MongoDB connection error: ${error.message}`);
18    process.exit(1);
19  }
20 };
21
22 module.exports = connectDB;
```

1. Configure MongoDB:

- Install Mongoose:
 - In your Node.js project directory, run:
 - `npm install mongoose`
- Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

1. Identity Resource Document Schema ([schemas/userModel.js](#))

Enforces field criteria validations on incoming database creations:

```
JS User.js  X
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2
3  const userSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: [true, 'Name is required'],
8        trim: true,
9      },
10     email: {
11       type: String,
12       required: [true, 'Email is required'],
13       unique: true,
14       lowercase: true,
15       trim: true,
16     },
17     password: {
18       type: String,
19       required: [true, 'Password is required'],
20       minlength: 6,
21     },
22   },
23   {
24     timestamps: true,
25   }
26 );
27
28 module.exports = mongoose.model('User', userSchema);
29
```

Project Executions:

- Install Postman or ThunderClient (VS Code extension).
- Start the backend server:
 - **npm start**

GitHub: [Code Repo Link](#)

Phase 6

Project Testing Phase

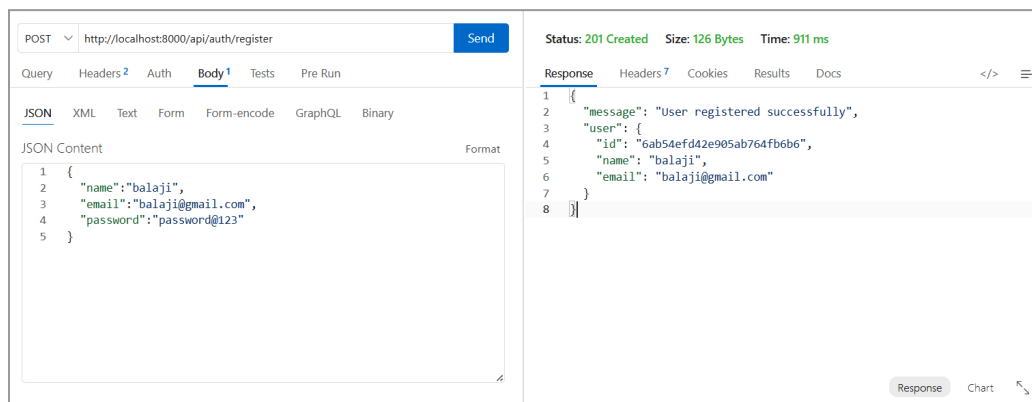
Phase 6: Project API Testing

Focuses on running the backend server and verifying all REST API endpoints using Postman or Thunder Client. It validates request payloads, JWT Bearer authorization, HTTP status codes (200 OK, 201 Created), and JSON responses across user registration, login, manual blog creation/fetching, AI blog generation, and AI content summarization.

User Registration:

HTTP Method: POST

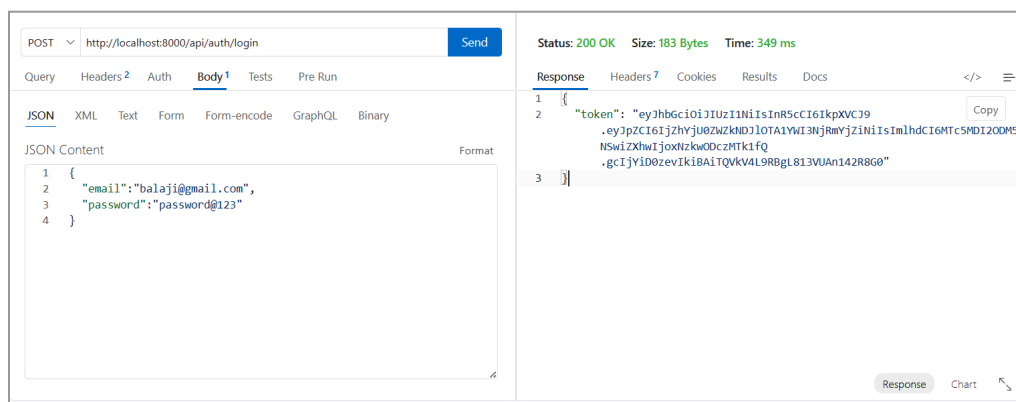
Endpoint: /api/auth/register



User Login:

HTTP Method: POST

Endpoint: /api/auth/login



Create Blog:

HTTP Method: POST

Endpoint: /api/blogs

The screenshot shows a REST client interface with a POST request to `http://localhost:8000/api/blogs`. The request body is a JSON object: `{ "title": "Marvel blog", "content": "Avengers Endgame Encore is Running Only on theaters", "category": "Superhero" }`. The response status is `201 Created` with a size of `287 Bytes` and a time of `509 ms`. The response body is a JSON object: `{ "title": "Marvel blog", "content": "Avengers Endgame Encore is Running Only on theaters", "category": "Superhero", "author": "6ab54efd42e905ab764fb6b6", "authorName": "balaji", "_id": "6ab54f7142e905ab764fb6ba", "createdAt": "2026-09-24T16:27:29.360Z", "updatedAt": "2026-09-24T16:27:29.360Z", "_v": 0 }`.

```
POST http://localhost:8000/api/blogs Send
```

Query Headers Auth **Body** 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "title": "Marvel blog",
3   "content": "Avengers Endgame Encore is Running Only on theaters"
4   ,
5   "category": "Superhero"
6 }
```

Status: 201 Created Size: 287 Bytes Time: 509 ms

Response Headers 7 Cookies Results Docs

```
1 {
2   "title": "Marvel blog",
3   "content": "Avengers Endgame Encore is Running Only on theaters",
4   "category": "Superhero",
5   "author": "6ab54efd42e905ab764fb6b6",
6   "authorName": "balaji",
7   "_id": "6ab54f7142e905ab764fb6ba",
8   "createdAt": "2026-09-24T16:27:29.360Z",
9   "updatedAt": "2026-09-24T16:27:29.360Z",
10  "_v": 0
11 }
```

Response Chart ↗

Fetch Blogs:

HTTP Method: GET

Endpoint: /api/blogs

The screenshot shows a REST client interface with a GET request to `http://localhost:8000/api/blogs`. The response status is `200 OK` with a size of `5 KB` and a time of `322 ms`. The response body is a JSON array containing one blog object: `[{ "_id": "6ab54f7142e905ab764fb6ba", "title": "Marvel blog", "content": "Avengers Endgame Encore is Running Only on theaters", "category": "Superhero", "author": { "_id": "6ab54efd42e905ab764fb6b6", "name": "balaji", "email": "balaji@gmail.com" }, "authorName": "balaji", "createdAt": "2026-09-24T16:27:29.360Z", "updatedAt": "2026-09-24T16:27:29.360Z", "_v": 0 }]`.

```
GET http://localhost:8000/api/blogs Send
```

Query Headers 2 Auth **Body** 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "title": "Marvel blog",
3   "content": "Avengers Endgame Encore is Running Only on theaters"
4   ,
5   "category": "Superhero"
6 }
```

Status: 200 OK Size: 5 KB Time: 322 ms

Response Headers 7 Cookies Results Docs

```
1 [
2   {
3     "_id": "6ab54f7142e905ab764fb6ba",
4     "title": "Marvel blog",
5     "content": "Avengers Endgame Encore is Running Only on
6     theaters",
7     "category": "Superhero",
8     "author": {
9       "_id": "6ab54efd42e905ab764fb6b6",
10      "name": "balaji",
11      "email": "balaji@gmail.com"
12    },
13     "authorName": "balaji",
14     "createdAt": "2026-09-24T16:27:29.360Z",
15     "updatedAt": "2026-09-24T16:27:29.360Z",
16     "_v": 0
17   }
18 ]
```

Response Chart ↗

Generate AI Blog:

HTTP Method: POST

Endpoint: /api/ai/generate-blog

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8000/api/ai/generate-blog
- Body:** JSON Content

```
1 {  
2   "topic": "Avengers",  
3   "category": "Cinema"  
4 }
```
- Status:** 201 Created
- Size:** 1.18 KB
- Time:** 7.55 s
- Response:**

```
1 {  
2   "title": "Guide to Avengers",  
3   "content": "When Earth's Mightiest Heroes first assembled in 2012, cinema changed forever. The Avengers did not just deliver blockbuster action; it redefined modern storytelling by weaving individual comic mythologies into one cohesive, colossal narrative. \n\nAt the heart of this phenomenon was the dynamic chemistry between wildly different personalities. From Tony Stark's razor-sharp wit and Steve Rogers' unwavering moral compass to Thor's cosmic grandeur, these flawed champions felt profoundly human. As the stakes escalated across a decade, culminating in the cultural event of Endgame, the franchise proved that superhero films could evoke genuine grief, triumph, and unity. The team resonated globally because they showed that our differences become our greatest strengths when facing insurmountable odds. \n\nLong after the final battle cries have faded, the legacy of the Avengers endures as a defining metaphor in pop cinema and contemporary culture."
```

Summarize Blogs:

HTTP Method: POST

Endpoint: /api/ai/summarize

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8000/api/ai/summarize
- Body:** JSON Content

```
1 {  
2   "content": "Avengers"  
3 }
```
- Status:** 200 OK
- Size:** 341 Bytes
- Time:** 4.06 s
- Response:**

```
1 {  
2   "summary": "The Avengers are Marvel's premier team of \"Earth's Mightiest Heroes,\" uniting legendary figures like Iron Man, Captain America, Thor, and the Black Widow. Putting aside their differences, this diverse group bands together to protect the world and the cosmos from catastrophic threats that no single hero could face alone."  
3 }
```

Phase 7

Project Documentation Phase

Phase 7: Project Documentation Phase

Phase	Covers
Brainstorming & Ideation Phase	Content management problem & proposed AI CMS solution
Requirement Analysis Phase	Software & hardware requirements
Project Design Phase	Backend layered architecture & AI service integration
Project Planning Phase	ER diagram, user flow, roles & MVC pattern
Project Development Phase	Backend setup, MongoDB connection, schemas & core code
Project Testing Phase	API endpoint testing & validation
Project Documentation Phase	Code structure, schemas & endpoints compilation
Project Demonstration Phase	Working video demo & live API execution

- **Problem & Requirements** – Explains the blog management and content creation challenges, the proposed AI BlogNest CMS solution, role-based usage, and required hardware/software.
- **Design & Architecture** – Describes the layered backend architecture, Entity-Relationship (ER) database structure, MVC pattern, user access roles (Admin, Editor, Author, Reader), and backend user flow.
- **Development & Implementation** – Covers VS Code project setup, Node.js/Express server configuration, MongoDB/Mongoose connection and schemas, JWT authentication, routing, and Gemini AI integration.
- **Testing & Demonstration** – Video demo covering backend setup and API testing for auth, blog management, and AI features.

Phase 8
Project Demonstration Phase

Phase 8: Project Demonstration Phase

The demonstration video provides a complete technical walkthrough of the AI BlogNest API, covering the codebase architecture, server initialization, and live endpoint verification.

- **Project File Explanation** – Walks through the VS Code workspace and src directory following the MVC pattern, explaining the database setup (config/db.js), data schemas (models), business logic (controllers), API route mappings (routes), JWT security and error handling (middleware), and the Google Gemini AI integration (services/geminiService.js).
- **Backend Execution** – Demonstrates configuring environment variables (.env), initializing the Express application (app.js and server.js), and starting the local backend server via npm start alongside an active MongoDB connection.
- **API Endpoint Testing** – Uses Thunder Client / Postman to showcase the exact request payloads, authorization headers, HTTP status codes, and JSON responses across all six core endpoints:
 - **User Registration (POST /api/auth/register)**: Sends a raw JSON body containing name, email, and password. The server hashes the password, creates the user record in MongoDB, and returns a 201 Created status with the generated user ID and profile details.
 - **User Login (POST /api/auth/login)**: Submits the registered email and password credentials. The backend validates the account and returns a 200 OK status along with a signed, stateless JWT token used for accessing protected routes.

- **Create Blog (POST /api/blogs):** Passes the JWT Bearer token in the Authorization header alongside a JSON payload with title, content, and category. The server links the post to the logged-in author and returns a 201 Created status with the saved blog document and timestamps.
- **Fetch Blogs (GET /api/blogs):** Queries the database for all stored posts and returns a 200 OK status with a JSON array of blog documents, automatically populated with the author's ID, name, and email.
- **Generate AI Blog (POST /api/ai/generate-blog):** Sends a JSON prompt containing a topic and category. The controller calls the Gemini AI service to synthesize a full-length, structured blog post, saves it directly to MongoDB under the authenticated author, and returns a 201 Created status.
- **Summarize Blogs (POST /api/ai/summarize):** Submits content text in the request body. The AI engine condenses the input and returns a 200 OK status with a concise summary abstract.

Demo Video: [Demo Video Link](#)
